

Design and Synthesis of 16-bit ALU and Generation of Optimized Netlist

INTRODUCTION:

Synthesis of 16-bit ALU is described in RTL using Verilog, which defines its arithmetic and logic functions based on control inputs. The Yosys tool processes this high-level behavioral code, translating it into an optimized, technology-specific gate-level netlist. This conversion from an abstract design to a structural blueprint of interconnected standard logic cells is a critical step that prepares the ALU for the final place-and-route stage of the digital design workflow.

APPLICATION:

A 16-bit ALU is the core computational unit in devices like CPUs, microcontrollers, and DSPs. As the central component of the datapath, it uses control signals to perform a wide range of arithmetic and logic operations. This versatility, consolidating many functions into a single unit, makes the ALU essential for executing any set of programmed instructions.

PROCEDURE TO CREATE A LAB:

Step 1: Set up Project Directory

- 1.1: Create directory for project: `mkdir alu16bit`
- 1.2: Open project directory: `cd alu16bit`

Step 2: Create Design file

- 2.1: Open gvim editor for design with command: `gvim alu16bit.v`

RTL Design Code:

```
module alu16bit (  
    input  [15:0] A, B,  
    input  [3:0]  ALU_Sel,    // Operation select  
    output reg [15:0] ALU_Out,  
    output reg CarryOut  
);
```

```
reg [16:0] tmp; // temporary for carry/borrow

always @(*) begin
    // Default assignments
    tmp = 17'h00000;
    ALU_Out = 16'h0000;
    CarryOut = 1'b0;

    case (ALU_Sel)
        4'b0000: tmp = A + B; // Addition
        4'b0001: tmp = A - B; // Subtraction
        4'b0010: tmp = A & B; // AND
        4'b0011: tmp = A | B; // OR
        4'b0100: tmp = A ^ B; // XOR
        4'b0101: tmp = ~(A | B); // NOR
        4'b0110: tmp = ~(A & B); // NAND
        4'b0111: tmp = ~(A ^ B); // XNOR
        4'b1000: tmp = A << 1; // Logical Shift
Left
        4'b1001: tmp = A >> 1; // Logical Shift
Right
        4'b1010: tmp = {A[14:0], A[15]}; // Rotate Left
        4'b1011: tmp = {A[0], A[15:1]}; // Rotate Right
        4'b1100: tmp = (A > B) ? 16'd1 : 16'd0; // Greater
comparison
        4'b1101: tmp = (A == B) ? 16'd1 : 16'd0; // Equal
comparison

        default: tmp = 17'h00000;
    endcase

    // Assign outputs
    ALU_Out = tmp[15:0];
    CarryOut = tmp[16];
end
endmodule
```

2.2: Save the file using command- :wq!

Step 3: Synthesize the Design with Yosys

invoke Yosys in a new terminal by the following command
yosys

3.1: Use the following command to Synthesize:

```
# Read the Verilog source file
read_verilog alu16bit.v

# Set the top-level module and check hierarchy
hierarchy -check -top alu16bit

# high level stuff
proc; opt; fsm; opt; memory; opt

# mapping to internal cell library
techmap; opt

# mapping flip-flops to mycells.lib
dfflibmap -liberty osu018_stdcells.lib

# mapping logic to mycells.lib
abc -liberty osu018_stdcells.lib

# Clean up unused cells and signals after mapping
clean

# Write the final synthesized gate-level netlist to a new
Verilog file
write_verilog alu16bit_synth.v
```

Step 4: View Output File (Netlist)

```
gvim alu16bit_synth.v
```

Observation :

- The ALU design is synthesized successfully without any errors.
- The arithmetic operations (like addition and subtraction) are correctly mapped into combinational adder/subtractor circuits. adder circuits.

- The bitwise logic functions (such as AND, OR, XOR) are implemented as parallel arrays of combinational logic gates.
- The control logic is synthesized correctly, using the function select lines to route the result from the appropriate operation to the final output, typically via a large multiplexer.

Output :

- Synthesized Netlist (alu16bit_synth.v)

Conclusion :

The synthesis output confirms the correct implementation of the 16-bit ALU, showing the RTL design was successfully mapped to a gate-level netlist. This netlist preserves the intended structure, with distinct combinational circuits for arithmetic and logic functions, all governed by a properly synthesized control logic.